

Table of Contents

1	C++ Program	2
1.1	Inputting	2
1.2	Preprocessing	2
1.3	Compiling	2
1.3.1	Frontend	2
1.3.2	Middleend	2
1.3.3	Backend	2
1.4	Assembling	2
1.5	Linking	2
1.5.1	Static Library Creating (Alternative)	3
1.5.2	Dynamic Library Creating (Alternative)	3
1.6	Executing (Reference 4)	3
1.6.1	Program Loading	3
1.6.2	Process Initializing	3
1.6.3	Process Terminating	4
1.6.4	Symbolic Reference Resolving	4
2	Java Program	4
2.1	Inputting	4
2.2	Compiling	4
2.3	Executing	4
2.3.1	Packaging (Alternative)	4
3	AMD64 Instruction Set Architecture	6
3.1	Register	6
4	System V Application Binary Interface	6

FOUND@T10N

1 C++ Program¹

1.1 Inputting

SOURCE FILE = SOURCE CODE + DIRECTIVES	[WINDOWS LINUX MACOS].CPP;
--	----------------------------

1.2 Preprocessing^[1]

TRANSLATION UNIT = PREPROCESSED SOURCE FILE	[WINDOWS].I; [LINUX MACOS].II;
✱ Process – including header files, macro expansion, and conditional compilation – preprocessor directives!	

1.3 Compiling^[2]

1.3.1 Frontend

INTERMEDIATE REPRESENTATION	
✱ Verify syntax and semantics through lexical, syntax, and semantic analysis	

1.3.2 Middleend

OPTIMIZED INTERMEDIATE REPRESENTATION	
✱ Perform optimizations that are independent of CPU architecture through reachability analysis, constant propagation, relocation of computation, etc.	

1.3.3 Backend

TARGET-DEPENDENT ASSEMBLY LANGUAGE SOURCE CODE	[WINDOWS LINUX MACOS].S;
✱ Perform optimizations that are specific for the CPU architecture through register allocation, instruction scheduling to keep parallel execution units busy	

1.4 Assembling^[3]

OBJECT FILE = METADATA + [MACHINE ∈ OBJECT] CODE	[WINDOWS COFF].OBJ; [LINUX ELF, MACOS MACH-O].O;
✱ Object code is usually relocatable, and not usually directly executable	
✱ Metadata contains linking or debugging information, such as stack unwinding information	
✱ Assembly code is translated to machine code and same machine code can be packaged in different object file formats	

1.5 Linking^[4]

EXECUTABLE FILE	[WINDOWS COFF PE].EXE; [LINUX ELF, MACOS MACH-O];
✱ Symbol resolution (static) – associating each symbol reference with exactly one symbol definition from the symbol tables of its input relocatable object files	
✱ Section relocation – associating a memory location with each symbol definition, and then modifying all of the references to those symbols so that they point to this memory location	
✱ Static library contributes its code to output and shared library generally contributes symbol or dependency information	

¹g++ -save-temps -masm=intel -o main main.cpp

1.5.1 Static Library Creating (Alternative)^[5]

STATICALLY LINKED LIBRARY	[WINDOWS].LIB; [LINUX, MACOS].A;
* Collection of object files, usually comes in the form of ar archive files, augmented with a symbol table that accelerates the search for the needed files	

1.5.2 Dynamic Library Creating (Alternative)^[6]

DYNAMICALLY LINKED LIBRARY	[PE].DLL; [ELF].SO; [MACH-O].DYLIB;
* Dynamic libraries and executables usually have the same format, so only one loader is needed, and an executable can be used as a shared library	

1.6 Executing (Reference 4)

1.6.1 Program Loading

- * Map file segments, in which $p_vaddr \equiv p_offset \pmod{pg_size}$ for efficient mapping, to virtual memory segments
- * Last **file page** of text segment may contain first page of data segment and last data page may contain file information not relevant to the running process to save space
- * Segments' addresses are adjusted to ensure each logical page in the address space has a single set of permissions as if each segment were complete and separate
- * End of the data segment requires special handling, zero out, for uninitialized data; thus, (irrelevant) information not in the logical memory page must be zeroed out
- * Executable file typically contains absolute code and shared object typically contains PIC

1.6.2 Process Initializing

- * `exec(BA_OS)` creates machine state **S** for new process
- * System finds path to program interpreter (usually dynamic linker); then loads, runs linker before passing control to main application!
- * System may create the memory image of the application program before passing control to the interpreter program
- * Interpreter program performs several tasks, i.e. symbolic reference resolving (reference 1.6.4) and initialization routines processing²
- * Interpreter program eventually transfer control to entry point specified in `AT_ENTRY` entry (`_start` by default)
- * Immediately before calling `_start`, `$rbp ← 0`, `$rdx ← function pointer` that the application should register with `atexit(BA_OS)`

PATH	LINUX PATH
/lib/ldx32.so.1	/libx32/ld-linux-x32.so.2
/lib/ld64.so.1	/lib64/ld-linux-x86-64.so.2

Figure 2: Program Interpreter

- * Language-specific startup code transforms **S** into state required by language, i.e. `_start` prepares `argc`, `argv`, `envp`, then calls `main()` in C++

NOTE At process creation, the system may transfer control to an interpreter program. In this case, it places either an `AT_EXECFD` or an `AT_PHDR` entry in the auxiliary vector

0xfffffffffffffff	Reserved system area ...
	...
	Dynamic segments
0x7ffffffff	Stack segment ...
	...
	Data segments
	...
0x400000	Text segments
0	Unmapped

Figure 1: Conventional Segment Arrangements

High Address>	Unspecified
Varies Length	Information block argument strings, environment strings, auxiliary information, ...
	Unspecified
1 8Byte	Null auxiliary vector entry
Each> 2 8Byte	Auxiliary vector entries ...
Eightbyte> 8B	0
Each> 1 8Byte	Environment pointers ...
8+8*argc+\$rsp	0
8+\$rsp	Argument pointers
\$rsp	Argument count
Low Address>	Undefined

Figure 3: Initial Process Stack

²Initialization functions are specified by `DT_INIT`, `DT_INIT_ARRAY` and `DT_PREINIT_ARRAY`

1.6.3 Process Terminating

- * When `main()` returns, its value is passed to `exit()`; if `exit()` is overridden and returns, `_exit()`, which must be immune to user interposition, terminates the process
- * When `exit()` is called, registered `atexit()` handlers are executed, followed by the runtime's termination processing, which invokes `DT_FINI_ARRAY` and `DT_FINI`

1.6.4 Symbolic Reference Resolving^{3,4}

- * Each shared object file in the process image has its own PLT, and control transfers to a PLT entry only from within the same object file

```
.PLT0:  pushq    GOT+8(%rip)    # GOT[1] - identifying information
        jmp     *GOT+16(%rip) # GOT[2] - dynamic linker's resolver
        nopl    0x0(%rax)

.PLT1:  jmp     *name1@GOTPCREL(%rip) # 16 bytes from .PLT0
        pushq   $index1
        jmp     .PLT0

.PLT2:  ...                  # 16 bytes from .PLT1
```

- * Initially GOT holds address of following `pushq` instruction, not real address of `name1`
- * `pushq` pushes a relocation index on stack, indexing into the relocation table defined by `DT_JMPREL` dynamic section entry
- * The designated relocation entry will have type `R_X86_64_JUMP_SLOT`, its offset will specify GOT entry used in previous `jmp` instruction, and it contains a symbol table index for appropriate symbol `name1`
- * When dynamic linker receives control, after `jmp GOT[2]`, it unwinds the stack, looks at designated relocation entry, finds symbol's value, stores `name1` actual address in GOT

2 Java Program^[7]

2.1 Inputting

SOURCE FILE = SOURCE CODE	[WINDOWS LINUX MACOS].JAVA;
---------------------------	-----------------------------

2.2 Compiling

BYTECODE ∈ OBJECT CODE	[WINDOWS LINUX MACOS].CLASS;
* Platform-independent bytecode	

2.3 Executing

- * **Class loading** load classes as needed, using parent delegation
- * **Bytecode verifying or linking** prepare allocates static fields with default values, and resolve symbolic references to their corresponding runtime entities
- * **Initializing** assign static fields their initialized values and execute static initialization blocks
- * **Garbage collecting** reclaim heap memory occupied by objects that are no longer reachable
- * **Runtime services and JIT compiling** interpreter executes JVM bytecode, while JIT compiler identifies frequently executed “hot” code and may compile it into native machine code, which can be stored in the code cache. Native calls go through JNI to reach C/C++ libraries

2.3.1 Packaging (Alternative)

ARCHIVES AND MODULES	[WINDOWS LINUX MACOS].JAR;
----------------------	----------------------------

³`LB_BIND_NOW` environment variable can change dynamic linking behavior that evaluates PLT entries before transferring control to program

⁴`R_X86_64_TLSDESC` relocations support lazy relocation using dedicated PLT, GOT entries given by `DT_TLSDESC_PLT`, `DT_TLSDESC_GOT` [??]

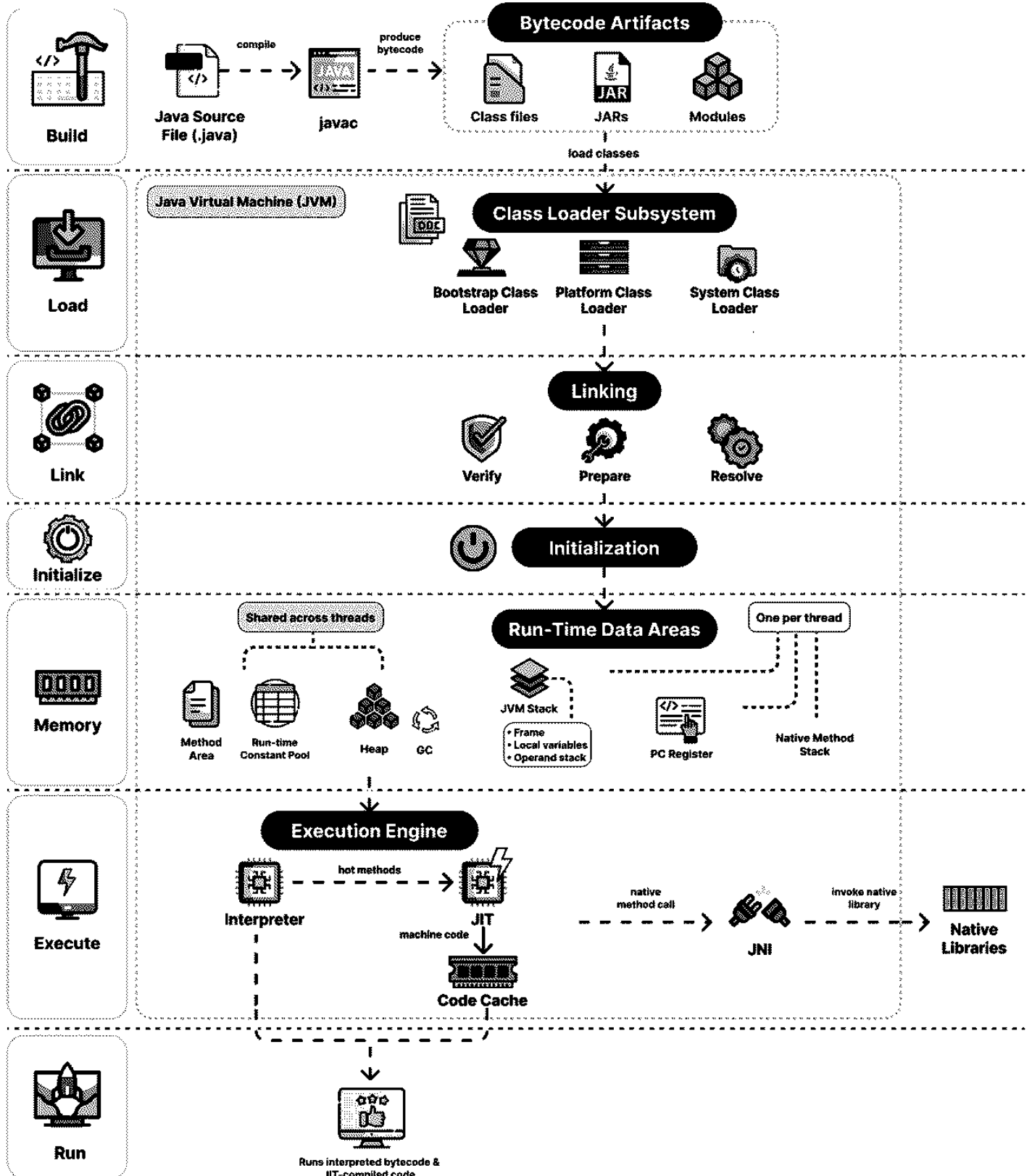


Figure 4: How the JVM Works

3 AMD64 Instruction Set Architecture (Reference 4)

3.1 Register

Field	Note	Register	Usage	
DF	Direction forward	\$rax	temporary register; with variable arguments passes information about the number of vector registers used; 1 st return register	T
CF	No carry			
PF	Even parity	\$rbx		
AF	No auxiliary carry	\$rcx	used to pass 4 th integer argument to functions	
ZF	No zero result	\$rdx	used to pass 3 rd argument to functions; 2 nd return register	T
SF	Unsigned result	\$rsp	stack pointer	
OF	No overflow occurred	\$rbp	optionally used as frame pointer	T
(a) rFLAGS Bits		\$rsi	used to pass 2 nd argument to functions	
		\$rdi	used to pass 1 st argument to functions	
FZ	Do not flush to zero	\$r8	used to pass 5 th argument to functions	
RC	Round to nearest	\$r9	used to pass 6 th argument to functions	
PM	Precision masked	\$r10	temporary register, used to pass a function's static chain pointer	
UM	Underflow masked	\$r11	temporary register	
OM	Overflow masked	\$r12-14		T
ZM	Zero divide masked	\$r15	optionally used as GOT base pointer	T
DM	De-normal operand masked	\$r16-31	temporary registers	
IM	Invalid operation masked	\$xmm0-1	used to pass and return floating point arguments	
DAZ	De-normals are not zero	\$xmm2-7	used to pass floating point arguments	
(b) MXCSR Status Bits		\$xmm8-15	temporary registers	
		\$xmm16-31	temporary registers	
		\$tmm0-7	temporary registers	
RC	Round to nearest	\$mm0-7	temporary registers	
PC	Double extended precision	\$k0-7	temporary registers	
PM	Precision masked	\$st0,st1	temporary registers, used to return long double arguments	
UM	Underflow masked	\$st2-7	temporary registers	
OM	Overflow masked	\$fs	thread pointer	T
ZM	Zero divide masked	mxcsr	SSE2 control [T] and status [F] word	P
DM	De-normal operand masked	x87 SW	x87 status word	
IM	Invalid operation masked	x87 CW	x87 control word	T
(c) x87 Floating-Point Control Word		tilecfg	Tile control register	

Figure 6: Special Registers

Figure 7: Register Usage: 3rd column indicates whether the register is non-volatile T denotes true, while P denotes partial

- * 16 general purpose 64-bit registers, 16 SSE 128-bit registers and 8 x87 floating point 80-bit registers (64-bit in MMX/3DNow!) are global to all procedures active for a given thread
- * Intel APX, Advanced Performance Extensions, provides 16 general purpose 64-bit registers \$r16-31
- * Intel AMX, Advanced Matrix Extensions, provides 8 tile registers \$tmmN
- * Intel AVX, Advanced Vector Extensions, provides 16 AVX 256-bit registers
- * Intel AVX-512 provides 32 SIMD 512-bit registers and 8 vector mask 64-bit \$kN
- * Vector register refers to either \$xmmN SSE, \$ymmN AVX or \$zmmN AVX-512 register (\$zmmN → \$ymmN → \$xmmN)

4 System V Application Binary Interface^[8]